Correcting Validity

Your task is to check the "productionStartYear" of the DBPedia autos datafile for valid values. The following things should be done:

- check if the field "productionStartYear" contains a year
- check if the year is in range 1886-2014
- convert the value of the field to be just a year (not full datetime)
- the rest of the fields and values should stay the same
- if the value of the field is a valid year in the range as described above, write that line to the output_good file
- if the value of the field is not a valid year as described above, write that line to the output_bad file
- discard rows (neither write to good nor bad) if the URI is not from dbpedia.org
- you should use the provided way of reading and writing data (DictReader and DictWriter) They will take care of dealing with the header.

You can write helper functions for checking the data and writing the files, but we will call only the 'process_file' with 3 arguments (inputfile, output_good, output_bad).

In [ ]:

```
import csv
import pprint

INPUT_FILE = 'autos.csv'
OUTPUT_GOOD = 'autos-valid.csv'
OUTPUT_BAD = 'FIXME-autos.csv'

def process_file(input_file, output_good, output_bad):

    with open(input_file, "r") as f:
        reader = csv.DictReader(f)
        header = reader.fieldnames

        #COMPLETE THIS FUNCTION



    # This is just an example on how you can use csv.DictWriter
    # Remember that you have to output 2 files
    with open(output_good, "w") as g:
        writer = csv.DictWriter(g, delimiter=",", fieldnames= header)
        writer.writeheader()
        for row in YOURDATA:
            writer.writerow(row)


def test():

    process_file(INPUT_FILE, OUTPUT_GOOD, OUTPUT_BAD)


if __name__ == "__main__":
    test()
```

#!/usr/bin/env python

#-- coding: utf-8 --

In this problem set you work with cities infobox data, audit it, come up with a cleaning idea and then clean it up. In the first exercise we want you to audit the datatypes that can be found in some particular fields in the dataset. The possible types of values can be:

- NoneType if the value is a string "NULL" or an empty string ""
- list, if the value starts with "{"
- int, if the value can be cast to int
- float, if the value can be cast to float, but CANNOT be cast to int. For example, '3.23e+07' should be considered a float because it can be cast as float but int('3.23e+07') will throw a ValueError
- 'str', for all other values

The audit_file function should return a dictionary containing fieldnames and a SET of the types that can be found in the field. e.g. {"field1": set([type(float()), type(int()), type(str())]), "field2": set([type(str())]), ... } The type() function returns a type object describing the argument given to the function. You can also use examples of objects to create type objects, e.g. type(1.1) for a float: see the test function below for examples.

Note that the first three rows (after the header row) in the cities.csv file are not actual data points. The contents of these rows should not be included when processing data types. Be sure to include functionality in your code to skip over or detect these rows.

In [ ]:

```
import codecs
import csv
import json
import pprint

CITIES = 'cities.csv'

FIELDS = ["name", "timeZone_label", "utcOffset", "homepage", "governmentType_label",
          "isPartOf_label", "areaCode", "populationTotal", "elevation",
          "maximumElevation", "minimumElevation", "populationDensity",
          "wgs84_pos#lat", "wgs84_pos#long", "areaLand", "areaMetro", "areaUrban"]

def audit_file(filename, fields):
    fieldtypes = {}

    # YOUR CODE HERE


    return fieldtypes


def test():
    fieldtypes = audit_file(CITIES, FIELDS)

    pprint.pprint(fieldtypes)

    assert fieldtypes["areaLand"] == set([type(float()),type(list()),type(None)])


if __name__ == "__main__":
    test()
```

Crossfield Auditing

In this problem set you work with cities infobox data, audit it, come up with a cleaning idea and then clean it up.

If you look at the full city data, you will notice that there are couple of values that seem to provide the same information in different formats: "point" seems to be the combination of "wgs84_pos#lat" and "wgs84_pos#long". However, we do not know if that is the case and should check if they are equivalent.

Finish the function check_loc(). It will recieve 3 strings: first, the combined value of "point" followed by the separate "wgs84_pos#" values. You have to extract the lat and long values from the "point" argument and compare them to the "wgs84_pos# values, returning True or False.

Note that you do not have to fix the values, only determine if they are consistent. To fix them in this case you would need more information. Feel free to discuss possible strategies for fixing this on the discussion forum.

Once you are done editig the code of the check_loc function, call the function process_file and examine the results.

In [ ]:

```
mport csv
mport pprint

ITIES = 'cities.csv'


ef check_loc(point, lat, longi):
    # YOUR CODE HERE

    pass


ef process_file(filename):
    data = []
    with open(filename, "r") as f:
        reader = csv.DictReader(f)
        #skipping the extra matadata
        for i in range(3):
            l = reader.next()
        # processing file
        for line in reader:
            # calling your function to check the location
            result = check_loc(line["point"], line["wgs84_pos#lat"], line["wgs84_pos#l
            if not result:
                print "{}: {} != {} {}".format(line["name"], line["point"], line["wgs8
            data.append(line)

    return data


ef test():
    assert check_loc("33.08 75.28", "33.08", "75.28") == True
    assert check_loc("44.57833333333333 -91.21833333333333", "44.5783", "-91.2183") ==

f __name__ == "__main__":
    test()
```

In [ ]: